

Implementazione parallela e analisi del BloomFilter

Simone Cappugi

16/04/2026



UNIVERSITÀ
DEGLI STUDI
FIRENZE

Tabella dei contenuti

1 Introduzione

1. Introduzione

2. Approccio di programmazione

3. Test e risultati: Costruzione Bloom filter con Processi

4. Test e risultati: Costruzione Bloom Filter con Thread

5. Test e risultati: Interrogazione Bloom Filter

Introduzione: Il BloomFilter

1 Introduzione

- **Obiettivo:**

- **Ottimizzazione della costruzione e interrogazione:** Identificare la strategia di parallelizzazione ottimale per gestire Filtri di Bloom
- **Superamento dei limiti sequenziali:** Ridurre il tempo proibitivo della fase di inserimento ("training") e aumentare il throughput delle query simultanee
- **Analisi:** Valutate diverse strategie (*Multiprocessing*, *Shared Memory*, *Joblib* e *Free-threading*) rispetto a una baseline sequenziale.

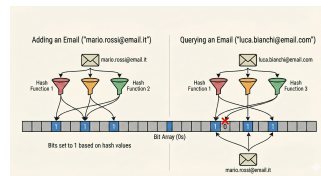


Figura: esempio di funzionamento

- **Generazione (faker):** libreria Faker (con localizzazione italiana) per simulare un ambiente di dati.
- Attraverso un EmailManager si possono generare email in due modalità
 - **Modalità Standard:** Generazione di email semplici nel formato nome.cognome@dominio
 - **Modalità Complessa:** Introduzione di maggiore entropia tramite suffissi casuali come numeri, anni o ruoli aziendali per testare il filtro in scenari più difficili

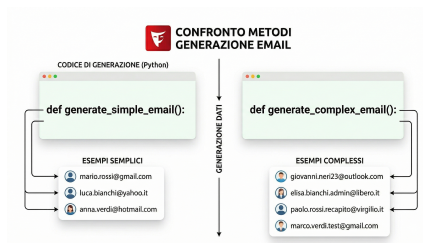


Figura: Esempio dei metodi

Obiettivi di sviluppo

1 Introduzione

- **Progettazione di un Filtro di Bloom Scalabile:**

Funzioni di hash MurmurHash3 (mmh3) per garantire velocità e distribuzione uniforme dei bit. Dimensionamento matematico ottimale dei parametri m (bit array) e k (numero di hash) in base alla probabilità di falso positivo desiderata ($P = 0.01$).

- **Analisi delle Prestazioni:**

Valutazione dello Speedup e del Throughput (query al secondo) al variare del numero di worker (da 1 a 16) e al variare della tecnica.

Obiettivi di sviluppo

1 Introduzione

- **Sperimentazione di Strategie di Parallelizzazione:**

Implementazione di modelli basati su Multiprocessing
(MapReduce e Shared Memory).

Esplorazione del paradigma Free-threading con Python 3.14 per
una gestione dei thread senza il blocco del GIL

MapReduce vs Shared Memory

1 Introduzione

Approccio MapReduce (Isolato)

- **Map:** Lavoro diviso in workers, ognuno elabora un *chunk* di dati in una memoria locale separata, generando un `bitarray` parziale.
- **Fase di Reduce:** Il Master aggrega i risultati finali tramite un operatore logico OR bit a bit (`|=`).
- **Vantaggi:** Architettura pulita, assenza di *race conditions* e alta efficienza con dataset medio-grandi.

Map Reduce vs Shared Memory

1 Introduzione

Approccio Shared Memory (Condiviso)

- **Meccanismo:** I worker scrivono simultaneamente in un unico blocco di memoria centrale (es. `multiprocessing.Array` o `NumPy Shared`)..
- **Sincronizzazione:** Approccio *lock-free* per scritture puramente additive, eliminando la fase di aggregazione finale.
- **Svantaggi:** Elevato overhead di setup IPC e potenziale degrado dovuto al *False Sharing* tra le cache dei core.

Tabella dei contenuti

2 Approccio di programmazione

1. Introduzione

2. Approccio di programmazione

3. Test e risultati: Costruzione Bloom filter con Processi

4. Test e risultati: Costruzione Bloom Filter con Thread

5. Test e risultati: Interrogazione Bloom Filter

Overview Algoritmo Sequenziale

2 Approccio di programmazione

Tre fasi logiche:

1. Pre-processing:

Lettura del dataset (CSV) e normalizzazione delle email (minuscolo, rimozione spazi/accenti).

2. Calcolo degli Hash:

Generazione di k indici di posizione tramite la funzione MurmurHash3 (mmh3) con *seed* diversi.

3. Inserimento (Training):

Accensione dei bit corrispondenti agli indici calcolati nell'array di dimensione m .

Struttura del loop sequenziale

```
# 1. Calcolo parametri ottimali m, k
# 2. Inizializzazione bit_array a 0
for raw_email in dataset
    # 3. Normalizzazione
    email = EmailManager.normalize(email);
    # 4. Hashing
    for idx in BloomFilter.calculate_hashes (email,m,k):
        # 5. Inserimento
        bit_array[idx] = 1
```

Strategie di Parallelizzazione: Orchestratore

2 Approccio di programmazione

- **Gestione Memoria:**

- **Isolata (MapReduce):** evita conflitti, ma richiede un *Merge* finale (OR logico).
- **Condivisa (Shared):** approccio *lock-free*, ma penalizzato dal setup IPC.

- **Scheduling e Worker:**

- **Multiprocessing:** Tecnica che permette di sfruttare più processi in parallelo.
- **Free-threading (No-GIL):** No-GIL, permette ai thread di funzionare. Assenza di overhead di serializzazione.

Multiprocessing: MapReduce vs. Shared Memory

2 Approccio di programmazione

Strategia MapReduce

```
# Master (orchestrator.py)
with Pool(n_workers) as pool:
    # MAP: Calcolo indipendente
    results = pool.imap_unordered(worker_task, args)
# REDUCE: Merge degli array locali
for local_ba in results:
    global_bit_array |= local_ba
```

Strategia Shared Memory

```
shm = Array('b', m, lock=False)
with Pool(initializer=init_shared, initargs=shm,
processes=n_workers) as pool:
    #Scrittura diretta in SHM
    pool.map(worker_shared, args)
# Memoria già aggiornata passaggio in bytearray
final_ba = bytearray(shm)
```

Nota: Nel Bloom Filter, la *Shared Memory* è **lock-free** poiché le operazioni sono puramente additive (impostazione di bit a 1).

Multiprocessing con Joblib

- **Backend Loky:** Gestione efficiente del pool di processi per il riutilizzo dei worker.
- **Vantaggi principali:**
 - Miglior speedup tra le varianti a processi (fino a **2.81x**).
 - Mitigazione dell'overhead di serializzazione tipico del multiprocessing standard.

Threaded (No-GIL)

- **Python 3.14:** La rimozione del *Global Interpreter Lock* parallelismo reale tra thread.
- **Zero-Copy:** Condivisione dello spazio di indirizzamento, eliminando il IPC.
- **Vantaggi principali:**
 - Speedup massimo assoluto (**3.4x**).
 - Eliminazione totale del *Pickling*.
 - Buona scalabilità quasi nelle query di interrogazione.

Tabella dei contenuti

3 Test e risultati: Costruzione Bloom filter con Processi

1. Introduzione

2. Approccio di programmazione

3. Test e risultati: Costruzione Bloom filter con Processi

4. Test e risultati: Costruzione Bloom Filter con Thread

5. Test e risultati: Interrogazione Bloom Filter

Dataset

3 Test e risultati: Costruzione Bloom filter con Processi

Generazione Dataset

- Come primo passo viene generato il dataset. Per l'esperimento sono stati utilizzati:
- dataset_10k.csv, dataset_100k.csv, dataset_500k.csv, dataset_1.5m.csv, dataset_3m.csv, dataset_5m.csv, dataset_10m.csv

Analisi dei Risultati (costruzione del bloom filter)

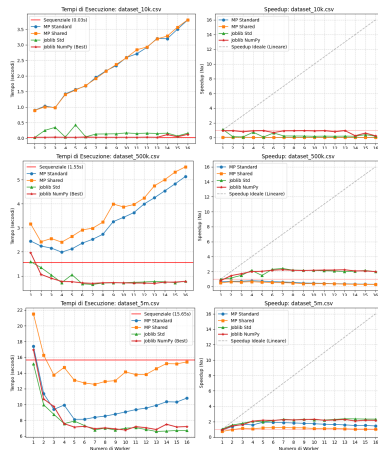
3 Test e risultati: Costruzione Bloom filter con Processi

Dataset 10k – 500k: Il muro dell'overhead

- **Effetto:** Speedup < 1 (peggio del sequenziale).
- **Causa:** Costi fissi di *startup* e IPC dominanti su un calcolo infinitesimo.

Dataset 5m: Scalabilità reale

- **Effetto:** Il carico computazionale "nasconde" i costi di comunicazione.



Analisi dei Risultati: Profilazione (1)

3 Test e risultati: Costruzione Bloom filter con Processi

Fase	Tempo (μs)	%	Operazione
Map-Reduce (Standard)			
Creazione Array	158.4	0.0%	bitarray(m)
Setup Pool	31k	2.6%	Pool(...)
Attesa Worker	1.16M	96.8%	for ba in results
Unione Finale	1.5k	0.1%	=
Shared Memory			
Allocazione IPC	4.7k	0.3%	Array('b', m)
Calcolo	1.17M	70.8%	pool.map(...)
Conversione	442k	26.6%	extend(...)

Il bottleneck principale è l'attesa dei worker e la fase di reduce.

Tabella dei contenuti

4 Test e risultati: Costruzione Bloom Filter con Thread

1. Introduzione

2. Approccio di programmazione

3. Test e risultati: Costruzione Bloom filter con Processi

4. Test e risultati: Costruzione Bloom Filter con Thread

5. Test e risultati: Interrogazione Bloom Filter

Analisi dei Risultati

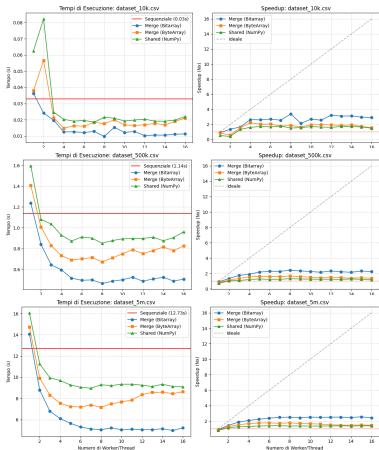
4 Test e risultati: Costruzione Bloom Filter con Thread

Dataset 10k - 500k:

- I thread sono più veloci del sequenziale già con 10k elementi .
- Assenza di *pickling* e velocità di *spawn* eliminano l'overhead iniziale.
- Il **Map Reduce** (threaded merge) risulta superiore allo *Shared*
- Lo *Shared* è penalizzato dal *False Sharing*.

Dataset 5m:

- Scaling limitato solo dalla saturazione hardware.



Analisi dei Risultati: Profilazione

4 Test e risultati: Costruzione Bloom Filter con Thread

Metriche di Profilazione

- **Setup & Split:** Partizionamento del dataset in *chunk* per i thread.
- **Calcolo Core:** *Hashing* e attivazione bit. È la fase che scala con l'hardware.
- **Fase Finale:** Unione dei risultati (*Merge* OR o Conversione *Shared*).

Fase (ms)	Merge	Shared
Setup & Split	4.53	4.37
Allocazione Memoria	—	0.05
Calcolo Thread (Core)	447.06	632.34
Fase Finale (Merge/Conv.)	0.99	103.42
TEMPO TOTALE	452.58	740.18

Profilazione, tecnica map reduce vs shared.

Tabella dei contenuti

5 Test e risultati: Interrogazione Bloom Filter

1. Introduzione

2. Approccio di programmazione

3. Test e risultati: Costruzione Bloom filter con Processi

4. Test e risultati: Costruzione Bloom Filter con Thread

5. Test e risultati: Interrogazione Bloom Filter

Interrogazione Parallela: Struttura del Test

5 Test e risultati: Interrogazione Bloom Filter

- **Baseline Sequenziale:** Riferimento su singolo core (limite inferiore).
- **Multithreading (No-GIL):** I thread condividono nativamente il bitarray senza overhead IPC.
- **Multiprocessing (SHM):** Uso di `shared_memory` per sistemi Python standard. Mappatura del bitarray come `numpy.ndarray` per i worker.

Multiprocessing + Shared Memory

Utilizzo di `concurrent.futures`

```
# Avvio dei Processi (Shared Memory)
with ProcessPoolExecutor(max_workers=n) as exe:
    futures = [exe.submit(worker_shm, shm.name, ...)]
    for chunk in chunks]

# Avvio dei Thread (No-GIL)
with ThreadPoolExecutor(max_workers=n) as exe:
    futures = [exe.submit(worker_query, bf, ...)]
    for chunk in chunks]

# Raccolta risultati parziali
total = sum(f.result() for f in as_completed(futures))
```

Interrogazione: Metriche e Correttezza

5 Test e risultati: Interrogazione Bloom Filter

Performance (Throughput)

- **Query Elementi Presenti:** Richiedono il calcolo di tutti i k hash e il controllo di ogni bit (costo computazionale massimo).
- **Query Elementi Assenti:** Ottimizzate tramite *early exit*; il controllo si interrompe al primo bit o trovato.

Unità di misura: **q/s** (Query/Secondo)

Analisi Qualitativa

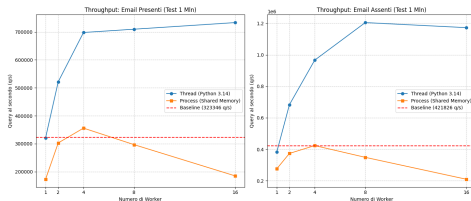
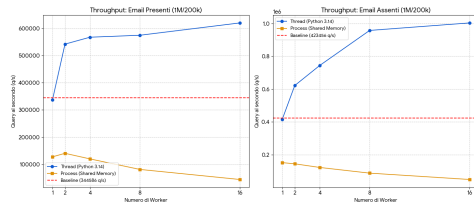
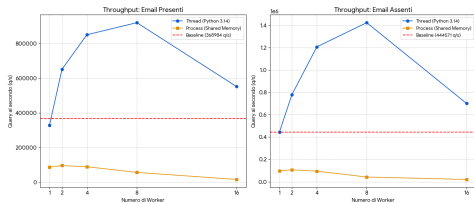
- **Zero Falsi Negativi:** Verifica l'integrità dei dati e l'assenza di bug nell'algoritmo o nella memoria condivisa.
- **Misurazione FPR:** Confronto tra il *False Positive Rate* reale e quello teorico calcolato in fase di design.

Correttezza

Il sistema conferma che il tasso di errore (FN) è pari a zero, garantendo la validità dei benchmark paralleli.

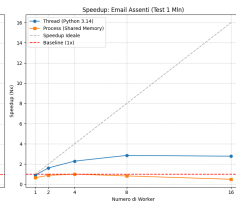
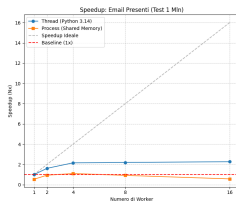
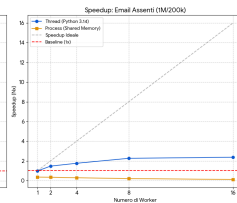
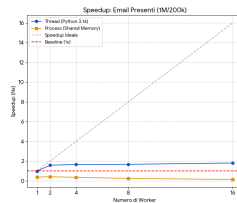
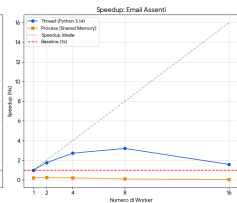
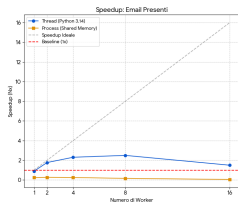
Analisi dei risultati - Throughput

5 Test e risultati: Interrogazione Bloom Filter



Analisi dei risultati - Speedup

5 Test e risultati: Interrogazione Bloom Filter



Dimostrazione delle Conclusioni: Profilazione

5 Test e risultati: Interrogazione Bloom Filter

- **Parità di Calcolo (Core):** Il tempo speso dalla CPU per hashing e query è identico ($\sim 217\text{ms}$).
- **Costo della Memoria (SHM):** L'allocazione e copia del bitarray nel SO richiede **312ms** per i processi (oms nei thread) .
- **Il muro del Pickling:** La serializzazione e l'invio delle email ai worker consuma **901ms** (62% del tempo totale). L'overhead IPC domina sul calcolo utile.

Fase dell'Esecuzione (ms)	Thread	Processi
Creazione SHM & Copia	—	312.27
Setup & Lancio Pool	3.24	4.74
Esecuzione Reale (Core)	217.78	217.17
Overhead (Pickling/Sync)	2.22	901.62
TEMPO TOTALE	223.23	1435.80

Il *Threading* vince annullando i costi di serializzazione e IPC.

Grazie per l' ascolto.